Chapter 5. Conversation Context와 Session 이해

학습 목표

이번 챕터에서는 Google ADK에서 대화가 어떻게 유지되는지 이해하고, **Conversation Context**와 **Session**의 개념을 학습합니다.

또한 Session을 활용하여 사용자의 이전 대화를 기억하는 AI Agent를 구현해 봅니다.

학습이 끝나면 다음 내용을 수행할 수 있습니다.

- Conversation Context의 개념 이해
- Session의 역할 이해
- Context와 Session의 관계 이해
- Google ADK의 Session 관리 방법 이해

이 장의 용어

용어	영문 · 약어 풀이	설명
Conversation Context	대화 문맥	현재 질문을 이해하기 위해 함께 전달되는 이전 대화 내용
턴	Turn	질문 한 번과 답변 한 번을 묶은 대화 단위. Single-turn은 1턴으로 끝나는 대화, Multi-turn은 여러 턴이 이어지는 대화
Context Window	컨텍스트 윈도우, 문맥 창	LLM이 한 번에 읽을 수 있는 최대 토큰 수. 넘으면 오래된 대화부터 제외됨
토큰	Token	LLM이 텍스트를 처리하는 최소 단위. Context Window의 크기는 토큰 수로 정해짐
Session	세션, 대화방	한 사용자와 Agent 사이의 대화를 묶어 관리하는 단위. Context를 담는 그릇
Session ID	세션 식별자	Session을 구분하는 고유한 값. 같은 ID를 쓰면 같은 대화가 이어짐
생명주기	Lifecycle	생성되어 사용되다가 종료되기까지의 전체 과정

용어	영문 · 약어 풀이	설명
Memory	메모리, 장기 기억	Session이 끝나도 유지되는 사용자 정보. Context와 구분
SessionService	세션 서비스	Google ADK에서 Session을 만들고 찾고 저장하는 구성 요소
컴포넌트	Component, 구성 요소	시스템을 이루는 독립된 기능 부품. 교체하거나 조합할 수 있음
In-Memory / RAM	Random Access Memory, 주기억장치	프로그램이 실행되는 동안만 데이터를 두는 임시 저장 공간. 종료하면 사라짐
영속성	Persistence	프로그램이 종료되어도 데이터가 남는 성질. 데이터베이스에 저장하면 확보됨
Redis / Firestore / MongoDB	—	Session을 영구 저장할 때 쓰는 저장소 종류. Redis는 빠른 메모리 기반 저장소, Firestore는 Google의 클라우드 DB, MongoDB는 문서형 DB
Runner	러너, 실행기	Agent와 SessionService를 연결해 대화를 실제로 돌리는 ADK 구성 요소. adk web도 내부에서 사용
비동기	Asynchronous, async/await	오래 걸리는 작업(LLM 응답 대기 등)을 기다리는 동안 다른 일을 할 수 있게 하는 실행 방식. Python에서는 `async def`, `await` 문법 사용

1. Conversation Context 이해

1.1. Conversation Context란?

사람과 사람의 대화를 생각해 보겠습니다.

다음과 같은 대화를 나누었다고 가정해 보겠습니다.

```
사용자 : 제 이름은 홍길동입니다.

AI : 안녕하세요, 홍길동님.

사용자 : 제 이름이 뭐였죠?
```

사람이라면 이전 대화를 기억하고 있기 때문에 쉽게 대답할 수 있습니다.

```
AI : 홍길동입니다.
```

하지만 이전 대화를 기억하지 못한다면 어떻게 될까요?

```
AI : 죄송하지만 알 수 없습니다.
```

이처럼 **이전 대화 내용을 기억하고 다음 대화에 활용하는 정보**를 **Conversation Context**라고 합니다.

> **Conversation Context는 현재 대화를 이해하기 위해 필요한 이전 대화와 관련 정보를 의미합니다.**

즉, AI Agent가 자연스럽게 대화를 이어가기 위해 반드시 필요한 정보입니다.

1.2. LLM과 Context

LLM(Large Language Model)은 기본적으로 **입력된 내용을 기반으로 다음 응답을 생성**합니다.

예를 들어 다음과 같은 질문을 입력하면

```
대한민국의 수도는 어디인가요?
```

LLM은

```
서울입니다.
```

라고 답변할 수 있습니다.

하지만 다음과 같이 질문하면 어떨까요?

```
그곳의 인구는 얼마인가요?
```

여기서 **'그곳'**이 무엇을 의미하는지는 이전 질문을 알아야 합니다.

즉, LLM은 현재 질문만으로는 의미를 정확하게 이해할 수 없습니다.

따라서 이전 대화를 함께 전달해야 합니다.

```
사용자 : 대한민국의 수도는 어디인가요?

AI : 서울입니다.
```

```
사용자 : 그곳의 인구는 얼마인가요?
```

이처럼 **현재 질문뿐만 아니라 이전 대화도 함께 전달하는 것**이 Conversation Context입니다.

1.3. Context가 필요한 이유

Context가 없다면 AI는 매번 새로운 대화라고 생각합니다.

예를 들어 다음과 같은 대화를 살펴보겠습니다.

Context가 없는 경우

```
사용자 : 저는 파이썬을 배우고 있습니다.

AI : 좋은 선택입니다.

사용자 : 얼마나 배우면 될까요?

AI : 무엇을 배우고 계신가요?
```

AI는 이전 질문을 기억하지 못하기 때문에 다시 질문하게 됩니다.

Context가 있는 경우

```
사용자 : 저는 파이썬을 배우고 있습니다.

AI : 좋은 선택입니다.

사용자 : 얼마나 배우면 될까요?

AI : 파이썬 기초는 보통 2~3개월 정도 학습하면 기본적인 문법을 익힐 수 있습니다.
```

이처럼 Context를 활용하면 훨씬 자연스럽고 사람과 같은 대화를 수행할 수 있습니다.

1.4. Single-turn과 Multi-turn 대화

Conversation Context를 이해하기 위해서는 **Single-turn**과 **Multi-turn** 대화의 차이를 알아야 합니다. 턴(Turn)은 질문 한 번과 답변 한 번을 묶은 대화 단위입니다.

Single-turn 대화

Single-turn은 **한 번의 질문과 한 번의 답변으로 끝나는 대화**입니다.

```
사용자

↓

오늘 날짜는?

↓

AI

↓

2026년 7월 6일입니다.
```

각 질문은 서로 독립적이며 이전 대화를 사용하지 않습니다.

Multi-turn 대화

Multi-turn은 **여러 번의 질문과 답변이 이어지는 대화**입니다.

```
사용자 : 오늘 서울 날씨 알려줘.

↓

AI : 맑고 30°C입니다.

↓

사용자 : 내일은?

↓

AI : 내일은 흐리고 28°C입니다.

↓

사용자 : 그 다음 날은?
```

여기서

> "내일은?"

> "그 다음 날은?"

이라는 질문은 이전 대화가 있어야 이해할 수 있습니다.

즉,

Multi-turn 대화에서는 Context가 반드시 필요합니다.

Single-turn과 Multi-turn 비교

구분	Single-turn	Multi-turn
이전 대화 사용	사용하지 않음	사용함
질문 독립성	높음	낮음
Context 필요	거의 없음	반드시 필요
대표 예시	검색	ChatGPT, AI Agent

실제 AI Agent는 대부분 Multi-turn 대화를 지원합니다.

1.5. Context Window란?

LLM은 모든 대화를 무한정 기억할 수 있는 것은 아닙니다.

모델에는 한 번에 처리할 수 있는 입력 길이에 제한이 있습니다.

이 범위를 **Context Window**라고 합니다.

쉽게 말하면,

> **LLM이 한 번에 읽고 이해할 수 있는 최대 대화 범위**입니다.

예를 들어 Context Window가 100이라고 가정하면

```
질문 1

질문 2

질문 3

...

질문 100
```

까지는 기억할 수 있지만,

새로운 대화가 계속 추가되면 오래된 대화는 제외될 수 있습니다.

> **참고**

> Context Window의 크기는 실제로는 질문 개수가 아니라 토큰(Token) 수 기준입니다.
> 위 예시는 이해를 돕기 위한 단순화입니다.

1.6. Context Window의 동작

예를 들어 다음과 같은 대화가 이어진다고 가정해 보겠습니다.

```
대화 1

↓

대화 2

↓

대화 3

↓

...

↓

대화 100
```

Context Window를 초과하면

```
대화 2

↓

대화 3

↓

...

↓

대화 101
```

처럼 가장 오래된 대화부터 제외됩니다.

즉, Context는 영구적으로 저장되는 것이 아니라 **일정 범위 안에서만 유지**됩니다.

1.7. Conversation Context와 Memory의 차이

Conversation Context는 현재 대화를 이어가기 위한 정보입니다.

반면 Memory는 장기간 유지되는 사용자 정보입니다.

예를 들어, Conversation Context

```
사용자 : 오늘 서울 날씨 알려줘.

↓

사용자 : 내일은?
```

현재 대화 안에서만 사용됩니다.

Memory

```
사용자는 Python 개발자이다.

사용자는 한국어로 답변받기를 선호한다.
```

다음 대화가 시작되어도 계속 사용할 수 있는 정보입니다.

이번 장에서는 **Conversation Context**만 다루며, Memory는 이후 심화 과정에서 학습합니다.

1.8. AI Agent에서 Context의 역할

AI Agent는 사용자의 이전 질문을 함께 전달하여 자연스러운 대화를 수행합니다.

전체 흐름은 다음과 같습니다.

```
사용자 질문

↓

이전 대화(Context)

↓

현재 질문

↓

LLM

↓
```

```
응답 생성
```

즉,

LLM은 **현재 질문만 보는 것이 아니라 이전 대화까지 함께 분석**하여 답변을 생성합니다.

실습 5-1. Context 유지 확인

다음 대화를 직접 실행해 봅니다.

```
사용자 : 제 이름은 홍길동입니다.

↓

AI : 안녕하세요.

↓

사용자 : 제 이름이 뭐죠?
```

Context가 유지된다면

```
홍길동입니다.
```

라고 응답합니다.

Context가 유지되지 않는다면

```
죄송하지만 이전 대화를 알 수 없습니다.
```

와 같은 응답을 하게 됩니다.

정리

이번 절에서는 Conversation Context의 개념과 역할을 학습했습니다.

개념	설명
Conversation Context	현재 대화를 이해하기 위한 이전 대화 정보
Single-turn	독립적인 한 번의 질문과 답변
Multi-turn	이전 대화를 활용하는 연속 대화

개념	설명
Context Window	LLM이 한 번에 처리할 수 있는 최대 대화 범위
Context의 역할	자연스럽고 연속적인 대화 지원

2. Session의 개념 이해

2.1. Session이란?

앞 절에서는 **Conversation Context**가 이전 대화 내용을 의미한다는 것을 학습했습니다.

하지만 여기서 한 가지 의문이 생깁니다.

> 여러 사용자가 동시에 AI Agent를 사용한다면, 누구의 대화를 기억해야 할까요?

예를 들어 다음과 같은 상황을 생각해 보겠습니다.

```
사용자 A : 제 이름은 홍길동입니다.

사용자 B : 제 이름은 김철수입니다.
```

잠시 후 두 사용자가 각각 질문합니다.

```
사용자 A : 제 이름이 뭐였죠?

사용자 B : 제 이름이 뭐였죠?
```

AI Agent는 사용자 A와 사용자 B의 대화를 서로 섞지 않고 기억해야 합니다.

이를 위해 사용하는 개념이 **Session**입니다.

> **Session은 하나의 사용자와 AI Agent 사이에서 이루어지는 대화를 관리하는 단위입니다.**

쉽게 말해 **"대화방"**이라고 생각하면 이해하기 쉽습니다.

각 사용자는 자신만의 대화방(Session)을 가지며, AI Agent는 해당 Session 안의 대화만 참고하여 응답을 생성합니다.

2.2. Session이 필요한 이유

Session이 없다면 모든 사용자의 대화가 하나로 섞이게 됩니다.

예를 들어 Session이 없는 경우를 살펴보겠습니다.

```
사용자 A : 제 이름은 홍길동입니다.

사용자 B : 제 이름은 김철수입니다.

사용자 A : 제 이름이 뭐였죠?
```

AI는 누구의 정보를 사용해야 하는지 알 수 없습니다.

반면 Session을 사용하면 다음과 같이 대화가 분리됩니다.

```
Session A

홍길동

↓

홍길동 관련 대화
```

```
Session B

김철수

↓

김철수 관련 대화
```

각 Session은 독립적으로 관리되므로 다른 사용자의 대화에 영향을 받지 않습니다.

2.3. Session ID의 역할

AI Agent는 여러 Session을 구분하기 위해 **Session ID**를 사용합니다.

Session ID는 각 Session을 식별하는 고유한 값입니다.

예를 들어 다음과 같이 두 개의 Session이 있다고 가정해 보겠습니다.

사용자	Session ID
사용자 A	session-001
사용자 B	session-002

사용자 A가 질문을 보내면

```
Session ID : session-001
```

을 함께 전달합니다.

그러면 AI Agent는

```
session-001

↓

기존 대화 조회

↓

Context 구성

↓

응답 생성
```

과정을 수행합니다.

즉, **Session ID는 어떤 대화를 이어갈 것인지를 결정하는 기준**입니다.

2.4. 사용자별 Session 관리

실제 AI 서비스에서는 동시에 수많은 사용자가 접속합니다.

예를 들어 다음과 같은 상황을 생각해 보겠습니다.

```
사용자 A

↓

Session A

↓

Conversation Context A
```

```
사용자 B

↓

Session B
```

```
↓

Conversation Context B
```

```
사용자 C

↓

Session C

↓

Conversation Context C
```

각 사용자는 자신의 Session 안에서만 대화를 이어갑니다.

따라서 사용자가 이전 질문을 기억하는 것처럼 느끼는 이유는 **같은 Session을 계속 사용하기 때문**입니다.

2.5. Session 생명주기(Lifecycle)

Session은 영구적으로 유지되는 것이 아닙니다.

생성되고, 사용되다가, 종료되는 생명주기를 가집니다.

일반적인 흐름은 다음과 같습니다.

```
Session 생성

↓

사용자 질문

↓

Context 저장

↓

추가 질문

↓

Context 갱신

↓

Session 종료
```

예를 들어 사용자가 처음 AI Agent에 접속하면 새로운 Session이 생성됩니다.

```
사용자 접속

↓

Session 생성
```

이후 질문을 보낼 때마다 Context가 추가됩니다.

```
질문 1

↓

질문 2

↓

질문 3
```

사용자가 로그아웃하거나 일정 시간이 지나면 Session이 종료될 수 있습니다.

2.6. AI Agent에서 Session의 역할

사용자가 질문을 입력하면 AI Agent는 다음과 같은 과정을 수행합니다.

```
사용자 질문

↓

Session ID 확인

↓

해당 Session 조회

↓

Conversation Context 구성

↓

LLM 호출

↓

응답 생성
```

```
↓

Context 저장
```

이 과정을 반복하면서 자연스러운 Multi-turn 대화가 이루어집니다.

2.7. Session과 Memory의 차이

Session은 **현재 대화를 관리하는 단위**입니다.

반면 Memory는 여러 Session에 걸쳐 장기간 유지되는 정보입니다.

예를 들어

```
Session

오늘 나눈 대화
```

은 Session이 종료되면 사라질 수 있습니다.

하지만

```
사용자는 Python 개발자이다.

사용자는 한국어를 선호한다.
```

와 같은 정보는 Memory에 저장되어 다음 Session에서도 활용될 수 있습니다.

정리

이번 절에서는 Session의 개념과 역할을 학습했습니다.

개념	설명
Session	하나의 사용자와 AI Agent 간의 대화를 관리하는 단위
Session ID	Session을 구분하는 고유한 식별자
Session 관리	사용자별 대화를 독립적으로 유지
Session 생명주기	생성 → 사용 → 종료
Context와의 관계	Session은 대화를 관리하고, Context는 그 안의 대화 내용

---

3. Context와 Session의 관계 이해

3.1. Context와 Session의 차이

앞 절에서는 Conversation Context와 Session을 각각 학습했습니다.

두 개념은 매우 밀접한 관계를 가지지만, 동일한 개념은 아닙니다.

많은 사람들이

> Context = Session

이라고 생각하지만 이는 잘못된 이해입니다.

두 개념의 역할은 다음과 같습니다.

구분	Context	Session
의미	이전 대화 내용	대화를 관리하는 단위
역할	AI가 이전 대화를 이해하도록 지원	사용자별 대화를 구분하고 관리
저장 대상	질문과 답변	Context, 사용자 정보 등
생성 시점	대화가 시작되면서 생성	사용자가 접속할 때 생성
종료 시점	Session 종료 시 함께 종료될 수 있음	Session 종료 시 종료

쉽게 표현하면 다음과 같습니다.

> **Session은 대화를 저장하는 공간이고, Context는 그 공간 안에 저장되는 대화 내용입니다.**

3.2. 비유로 이해하기

Context와 Session의 관계를 노트에 비유해 보겠습니다.

```
노트(Session)

├── 1페이지
│   사용자 : 안녕하세요.
│   AI : 안녕하세요.
│
├── 2페이지
│   사용자 : 제 이름은 홍길동입니다.
│   AI : 반갑습니다.
│
```

```
└── 3페이지
    사용자 : 제 이름이 뭐였죠?
```

여기서

- **노트 자체**가 Session입니다.
- **노트 안의 내용**이 Conversation Context입니다.

노트가 없으면 내용을 기록할 수 없고,

내용이 없으면 노트는 비어 있게 됩니다.

즉, Session과 Context는 서로 다른 역할을 수행하지만 함께 동작합니다.

3.3. Session 안에서 Context가 관리되는 방식

사용자가 처음 AI Agent에 접속하면 새로운 Session이 생성됩니다.

```
사용자 접속

↓

Session 생성
```

처음에는 Context가 비어 있습니다.

```
Session

└── Context : 없음
```

첫 번째 질문을 입력합니다.

```
사용자

안녕하세요.
```

그러면

```
Session

└── Context

    사용자 : 안녕하세요.
```

```
    AI : 안녕하세요.
```

와 같이 Context가 저장됩니다.

두 번째 질문을 입력합니다.

```
사용자

제 이름은 홍길동입니다.
```

그러면 Context는 다음과 같이 확장됩니다.

```
Session

└── Context

    사용자 : 안녕하세요.

    AI : 안녕하세요.

    사용자 : 제 이름은 홍길동입니다.

    AI : 반갑습니다.
```

질문과 답변이 계속 추가되면서 Context가 점점 커집니다.

3.4. AI는 어떻게 이전 대화를 기억할까?

AI는 사람처럼 영구적인 기억을 가지고 있는 것이 아닙니다.

질문이 들어올 때마다 Session에 저장된 Context를 함께 전달받습니다.

예를 들어

```
사용자

제 이름이 뭐였죠?
```

라는 질문이 들어오면

AI에게는 실제로 다음과 같은 정보가 전달됩니다.

```
사용자 : 안녕하세요.

AI : 안녕하세요.

사용자 : 제 이름은 홍길동입니다.

AI : 반갑습니다.

사용자 : 제 이름이 뭐였죠?
```

LLM은 이 전체 내용을 읽고

```
홍길동입니다.
```

라고 답변합니다.

즉, AI가 기억하는 것이 아니라

Session이 Context를 제공하기 때문에 기억하는 것처럼 보이는 것입니다.

3.5. 대화 흐름 예제

Conversation Context가 어떻게 이어지는지 전체 흐름을 살펴보겠습니다.

첫 번째 질문

```
사용자

오늘 서울 날씨 알려줘.
```

↓

```
AI

서울은 현재 맑고 29℃입니다.
```

두 번째 질문

```
사용자
```

```
내일은?
```

AI는

```
오늘 서울 날씨 알려줘.
```

라는 이전 Context를 함께 전달받기 때문에

```
내일 서울은 흐리고 27°C입니다.
```

라고 자연스럽게 응답할 수 있습니다.

세 번째 질문

```
사용자

우산이 필요할까?
```

AI는

현재까지의 모든 Context를 함께 고려하여

```
내일 비 예보가 있으므로 우산을 준비하는 것이 좋습니다.
```

와 같이 답변합니다.

3.6. 새로운 Session이 생성되면?

이번에는 새로운 Session을 생성해 보겠습니다.

새로운 Session은 기존 Session과 완전히 독립적입니다.

예를 들어

Session A

```
사용자

제 이름은 홍길동입니다.
```

↓

```
AI

반갑습니다.
```

이후

새로운 Session B를 생성합니다.

```
사용자

제 이름이 뭐였죠?
```

AI는

```
죄송하지만 이전 대화를 알 수 없습니다.
```

라고 답변합니다.

왜냐하면

Session B에는 아무런 Context가 저장되어 있지 않기 때문입니다.

3.7. Session 변경에 따른 Context 변화

다음 그림을 살펴보겠습니다.

```
Session A

└── Context

    홍길동

    Python

    서울 여행

    ↓

     새로운 Session 생성


Session B
```

```
└── Context

    (비어 있음)
```

새로운 Session에서는 처음부터 대화를 시작하게 됩니다.

3.8. AI Agent 내부 동작 구조

사용자가 질문을 입력하면 AI Agent 내부에서는 다음과 같은 과정이 수행됩니다.

```
① 사용자 질문

↓

② Session ID 확인

↓

③ Session 조회

↓

④ Conversation Context 조회

↓

⑤ Context + 현재 질문 결합

↓

⑥ LLM 호출

↓

⑦ 응답 생성

↓

⑧ Context 업데이트

↓

⑨ Session 저장
```

여기서 가장 중요한 과정은

```
Context + 현재 질문
```

입니다.

LLM은 현재 질문만 보는 것이 아니라, Session에서 가져온 Context와 함께 분석하여 응답을 생성합니다.

3.9. Context는 계속 커질까?

Context는 질문과 답변이 반복될수록 계속 증가합니다.

```
질문 1

↓

질문 2

↓

질문 3

↓

...

↓

질문 N
```

하지만 LLM은 한 번에 처리할 수 있는 Context Window가 제한되어 있습니다.

따라서 오래된 대화는 제거되거나 요약되어 관리될 수 있습니다.

Google ADK 역시 Session과 Context를 효율적으로 관리하기 위한 다양한 기능을 제공합니다.

3.10. 실무에서는 어떻게 활용될까?

실제 AI 서비스에서는 사용자마다 독립적인 Session을 생성합니다.

예를 들어 고객 상담 AI를 생각해 보겠습니다.

```
고객 A

↓

Session A

↓
```

```
주문 문의
```

```
고객 B

↓

Session B

↓

환불 문의
```

```
고객 C

↓

Session C

↓

배송 문의
```

각 고객은 자신의 Session 안에서만 대화를 이어가기 때문에 다른 고객의 정보가 섞이지 않습니다.

이러한 구조 덕분에 AI Agent는 수많은 사용자를 동시에 지원하면서도 개인화된 대화를 제공할 수 있습니다.

정리

이번 절에서는 Context와 Session이 어떻게 함께 동작하는지 학습했습니다.

개념	설명
Session	사용자별 대화를 관리하는 공간
Context	Session 안에 저장되는 대화 내용
Context 관리	질문과 답변이 계속 누적됨
새로운 Session	새로운 Context로 시작
AI 동작	Session에서 Context를 가져와 현재 질문과 함께 LLM에 전달

---

4. Google ADK의 Session 관리

4.1. SessionService란?

Google ADK에서 **SessionService**는 Session을 생성하고 관리하는 핵심 컴포넌트(Component, 독립된 기능 부품)입니다.

AI Agent는 SessionService를 통해 다음 작업을 수행합니다.

- 새로운 Session 생성
- 기존 Session 조회
- Session 삭제
- 대화 Context 저장 및 업데이트

구조는 다음과 같습니다.

```
사용자

↓

Agent

↓

SessionService

↓

Session

↓

Conversation Context
```

즉, SessionService는 사용자와 AI Agent 사이에서 **대화 상태를 유지하는 중간 관리자 역할**을 수행합니다.

4.2. InMemorySessionService

Google ADK는 다양한 저장 방식을 지원할 수 있도록 설계되어 있으며, 기본 구현체로 다음을 제공합니다.

```
InMemorySessionService
```

이 서비스는 Session을 메모리(RAM, Random Access Memory: 프로그램이 실행되는 동안만 데이터를 두는 임시 저장 공간)에 저장합니다.

특징은 다음과 같습니다.

- 설정이 매우 간단하다.
- 별도의 데이터베이스가 필요 없다.
- 테스트 및 학습에 적합하다.
- 프로그램 종료 시 모든 Session이 사라진다.

4.3. 실무에서는 어떻게 사용할까?

이번 장의 실습에서는 `InMemorySessionService`를 사용하지만, 실제 서비스에서는 영속성(Persistence, 프로그램이 종료되어도 데이터가 남는 성질)이 필요합니다.

따라서 다음과 같은 저장소를 활용합니다.

- MySQL
- PostgreSQL
- Redis
- Firestore
- MongoDB

이러한 저장소를 사용하면 서버가 재시작되어도 Session과 Context가 유지됩니다.

4.4. 실습 5-2. adk web에서 Session 확인하기

가장 간단한 실습은 `adk web`을 이용하는 것입니다.

`adk web`은 대화방 하나가 곧 하나의 Session입니다.

프로젝트 루트에서 Agent를 실행합니다.

```
adk web
```

다음 순서로 대화해 봅니다.

```
사용자 : 제 이름은 홍길동입니다.
```

```
AI : 반갑습니다, 홍길동님.

사용자 : 제 이름이 뭐였죠?

AI : 홍길동입니다.
```

같은 Session 안에서는 이전 대화(Context)가 유지되는 것을 확인할 수 있습니다.

이번에는 화면 왼쪽 상단의 **+ New Session** 버튼을 눌러 새로운 Session을 생성한 후 같은 질문을 해봅니다.

```
사용자 : 제 이름이 뭐였죠?

AI : 죄송하지만 이전 대화를 알 수 없습니다.
```

새로운 Session에는 Context가 없기 때문에 이름을 기억하지 못합니다.

4.5. 실습 5-3. 코드로 Session 관리하기 (선택)

이번에는 `adk web` 없이 Python 코드로 직접 Session을 생성하고 대화를 실행해 보겠습니다.

지금까지 비워 두었던 `main.py`를 사용합니다.

`main.py`

```python
import asyncio

from dotenv import load_dotenv
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types

from agent.agent import root_agent

load_dotenv()

APP_NAME = "my-adk"
USER_ID = "user-001"


async def main():
```

```
    # ① Session 저장소 생성
    session_service = InMemorySessionService()

    # ② 새로운 Session 생성
    session = await session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
    )

    # ③ Agent를 실행하는 Runner 생성
    runner = Runner(
        agent=root_agent,
        app_name=APP_NAME,
        session_service=session_service,
    )

    print("대화를 시작합니다. '종료'를 입력하면 끝납니다.")

    while True:
        user_input = input("사용자 : ")

        if user_input == "종료":
            break

        # ④ 사용자 입력을 메시지로 변환
        message = types.Content(
            role="user",
            parts=[types.Part(text=user_input)],
        )

        # ⑤ 같은 session_id로 실행하면 Context가 유지됩니다.
        async for event in runner.run_async(
            user_id=USER_ID,
            session_id=session.id,
            new_message=message,
        ):
            if event.is_final_response() and event.content:
                print("AI :", event.content.parts[0].text)
```

```
asyncio.run(main())
```

코드 핵심 살펴보기

`async` / `await` 는 비동기(Asynchronous) 실행 문법입니다. LLM 응답처럼 기다리는 시간이 긴 작업을 처리할 때 쓰며, 마지막 줄의 `asyncio.run(main())` 이 비동기 함수 `main` 을 실행합니다. 이번 과정에서는 "ADK의 Runner는 이 문법으로 호출한다" 정도만 알고 넘어가면 됩니다.

`InMemorySessionService()`

Session을 메모리에 저장하는 저장소를 생성합니다.

`create_session(...)`

새로운 Session을 생성합니다.

생성된 Session의 `session.id` 가 곧 Session ID입니다.

`Runner(...)`

Agent, Session 저장소를 연결하여 대화를 실행하는 관리자입니다.

`adk web` 도 내부적으로는 이 Runner를 사용합니다.

`runner.run_async(...)`

같은 `session_id` 를 계속 전달하기 때문에 이전 대화(Context)가 유지됩니다.

4.6. 실행 및 확인

터미널에서 실행합니다.

```
python main.py
```

다음 시나리오로 대화해 봅니다.

```
사용자 : 제 이름은 홍길동입니다.

AI : 반갑습니다, 홍길동님.

사용자 : 제 이름이 뭐였죠?

AI : 홍길동입니다.
```

```
사용자 : 종료
```

같은 Session ID를 사용했기 때문에 Agent가 이름을 기억합니다.

이번에는 프로그램을 **다시 실행**한 후 바로 이름을 물어봅니다.

```
사용자 : 제 이름이 뭐였죠?

AI : 죄송하지만 이전 대화를 알 수 없습니다.
```

`InMemorySessionService`는 메모리에만 저장하기 때문에 프로그램이 종료되면 모든 Session이 사라집니다.

실제 서비스에서 대화를 계속 유지하려면 4.3에서 소개한 데이터베이스 기반 저장소가 필요합니다.

정리

이번 절에서는 Google ADK의 Session 관리 기능을 실습했습니다.

구성 요소	역할
`SessionService`	Session의 생성, 조회, 저장을 담당
`InMemorySessionService`	메모리 기반 Session 저장소 (학습용)
`create_session()`	새로운 Session 생성
`Runner`	Agent와 Session을 연결하여 대화 실행
`session_id`	같은 값을 유지하면 Context가 이어짐

체크포인트 질문

1. "대한민국의 수도는?" 다음에 "그곳의 인구는?"이라고 물었을 때 AI가 "그곳"을 이해하려면 무엇이 필요한가요? → **이전 대화 내용, 즉 Conversation Context**
2. Session과 Context의 관계를 한 문장으로 설명하면? → **Session은 대화를 저장하는 공간이고, Context는 그 공간 안에 저장되는 대화 내용**
3. 사용자 A와 B의 대화가 서로 섞이지 않도록 구분해 주는 값은 무엇인가요? → **Session ID**
4. adk web에서 "+ New Session"을 누른 뒤 "제 이름이 뭐였죠?"라고 물으면 AI가 기억하지 못하는 이유는? → **새 Session에는 Context가 비어 있기 때문**

5. `InMemorySessionService`를 사용하는 프로그램을 종료 후 다시 실행하면 이전 대화가 남아 있나요? → **남아 있지 않음. 메모리에만 저장되므로 종료 시 사라짐. 유지하려면 데이터베이스 기반 저장소가 필요함**